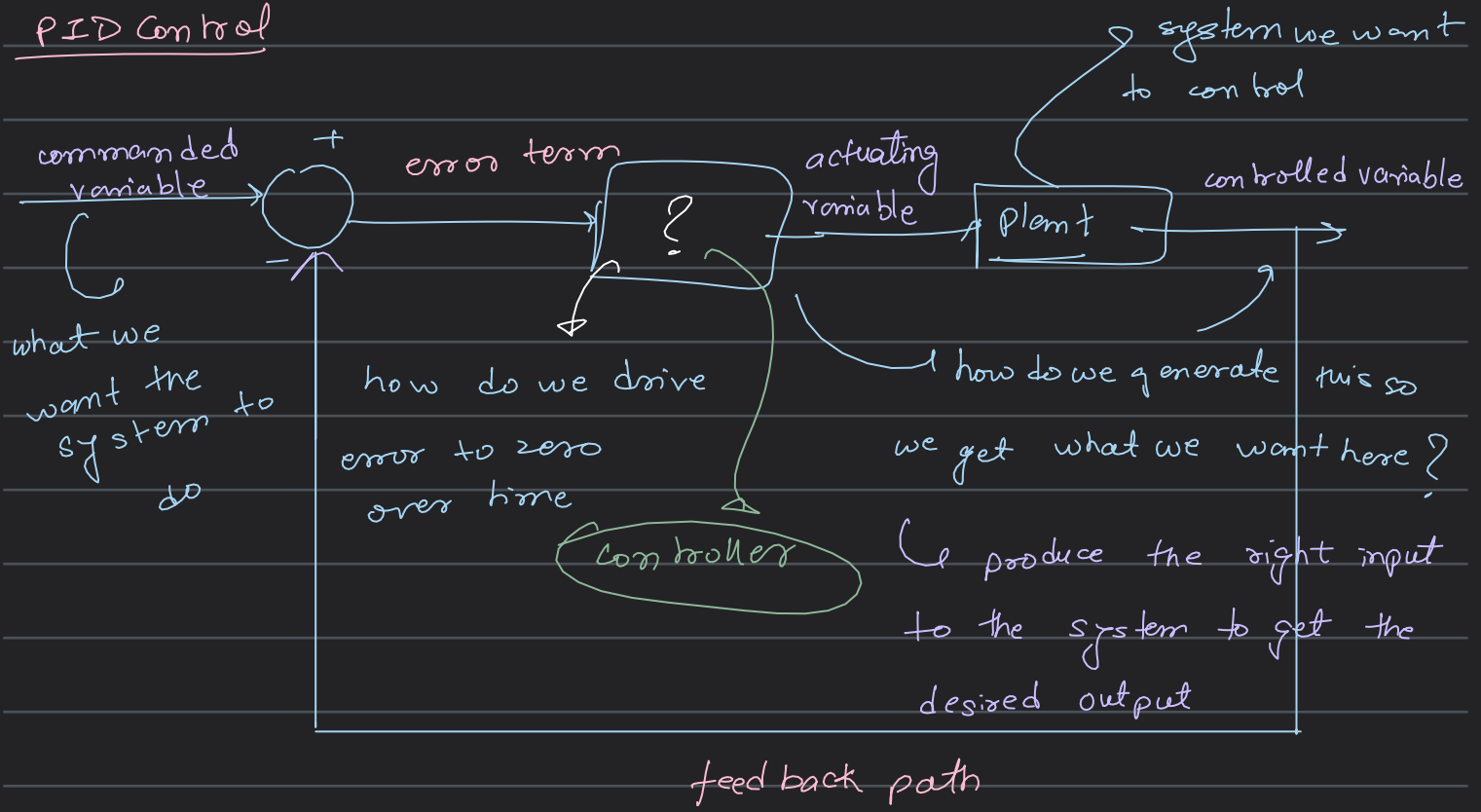


PPC -checkpoint-3

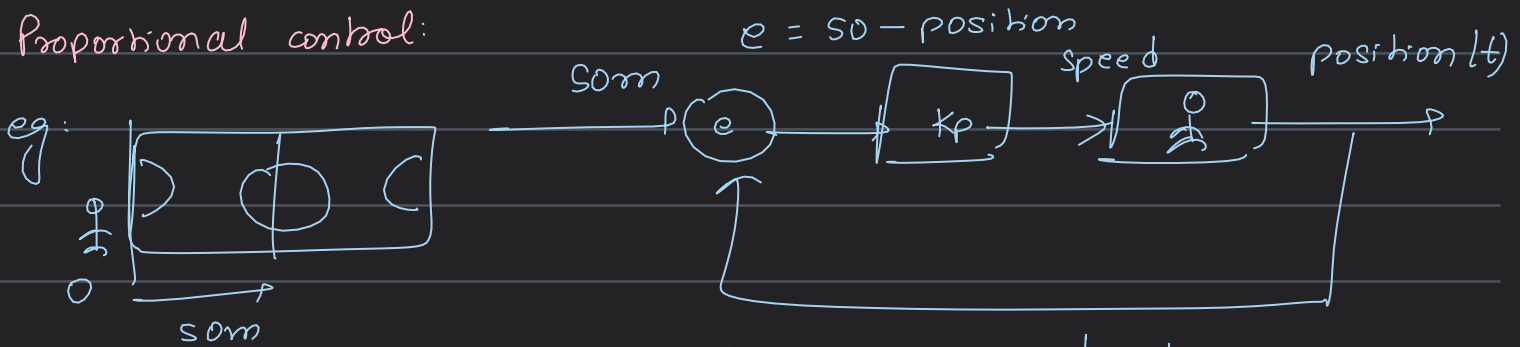
PID Control



error term = commanded (desired output) variable - controlled variable (output we got)

* we want zero error

Proportional control:

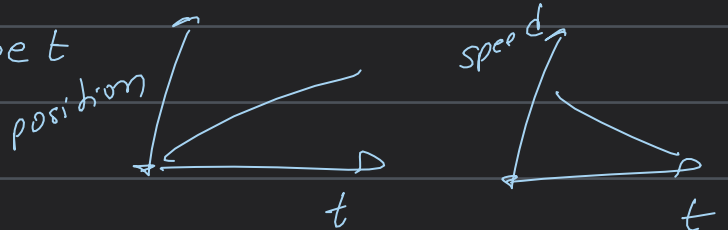


at $t=0$ $e = s_0 - 0$ $\text{speed} \propto e \Rightarrow \text{speed} = K_p e$

$\text{position} = K_p e t$ $e = s_0 - K_p e t$

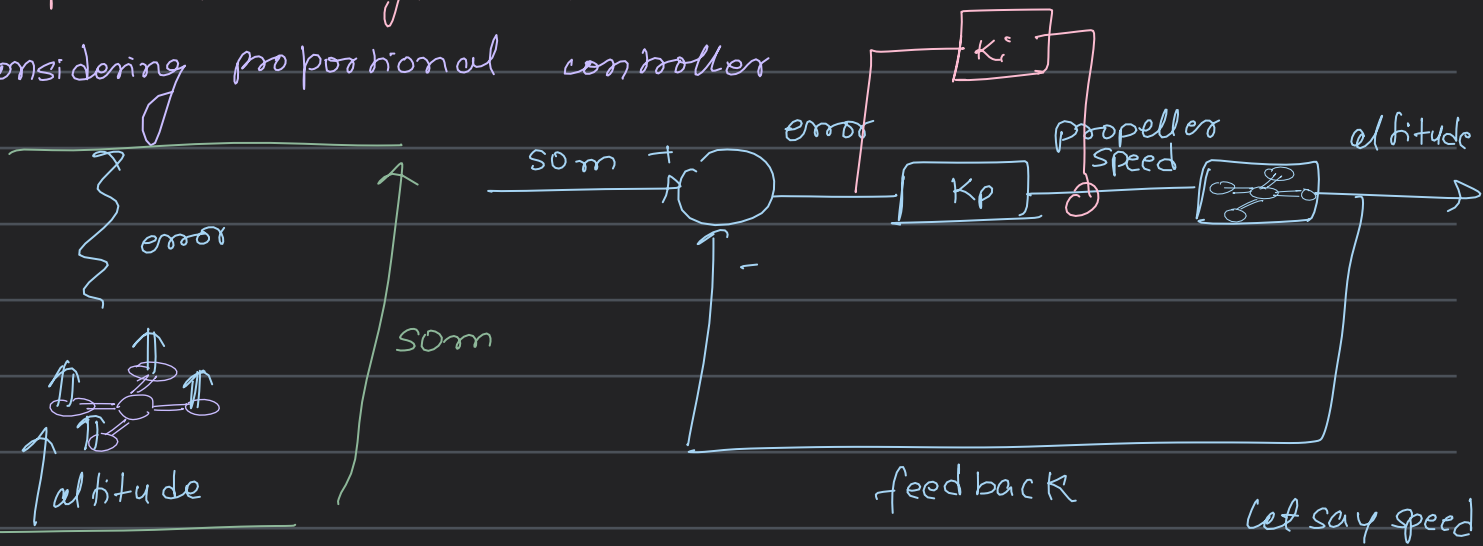
this problem can be solved

using the proportional control



Proportional-Integral Controller

Considering proportional controller



$$\text{Speed} = K_p e$$

$$\text{error} \times K_p = \text{Propeller speed}$$

Let say speed is 100rpm

steady state error

$$50 \times 2 = 100$$

the error never

$$20 \times 5 = 100$$

becomes zero
just keeps on dec

$$10 \times 10 = 100$$

$$1 \times 100 = 100$$

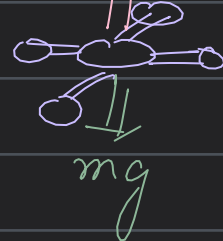
when we make $e=0$

drone shut

free fall

till the point

propeller lift

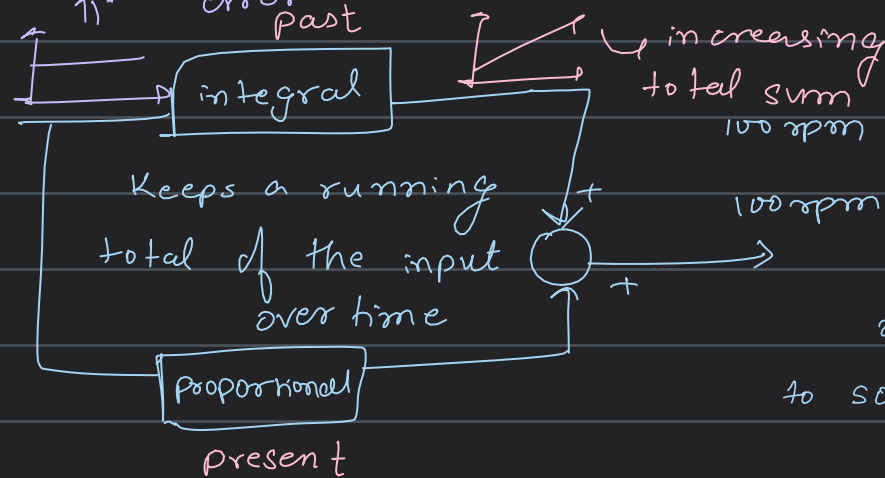


the drone will keep hovering at the altitude of e_p^m

to solve this problem, use past info

PI controller

steady state error past



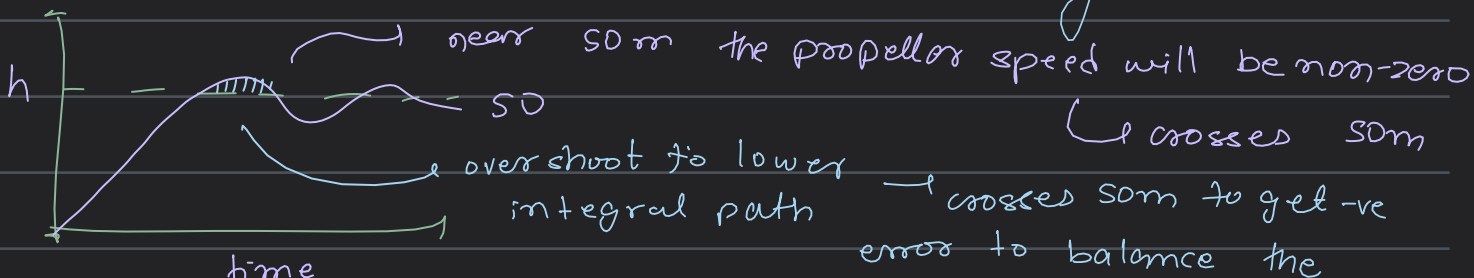
(output inc)

the propellers will move faster $\Rightarrow$ the drone will

rise up from its e_p^m state

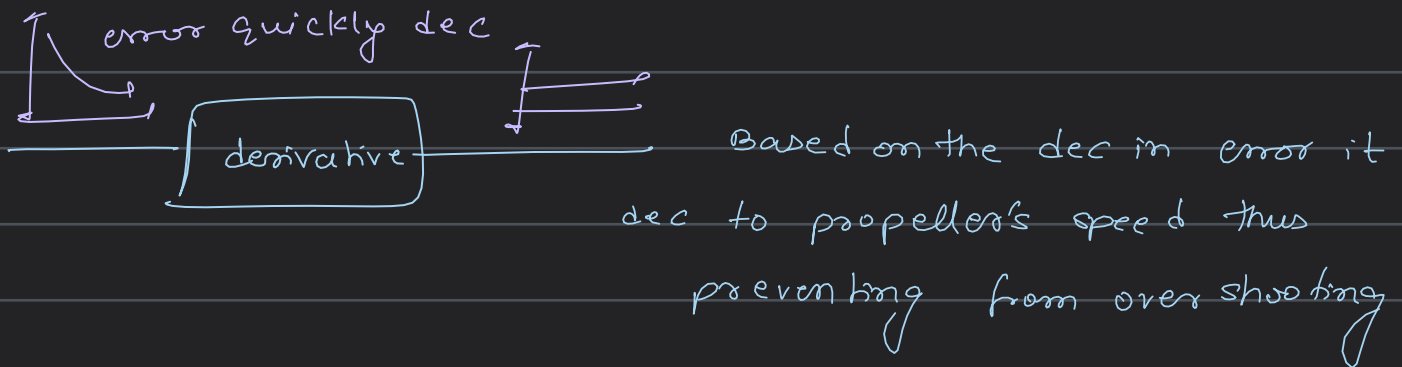
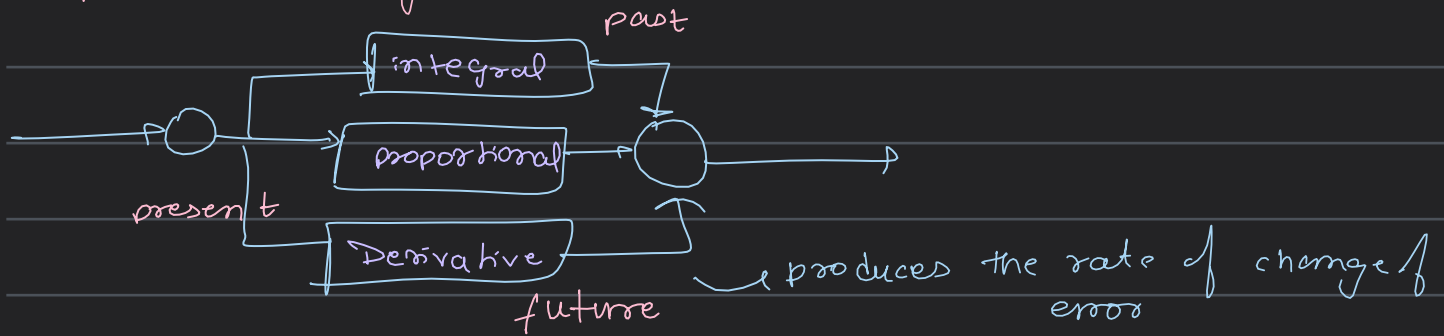
to 50m with the integral controller working

working



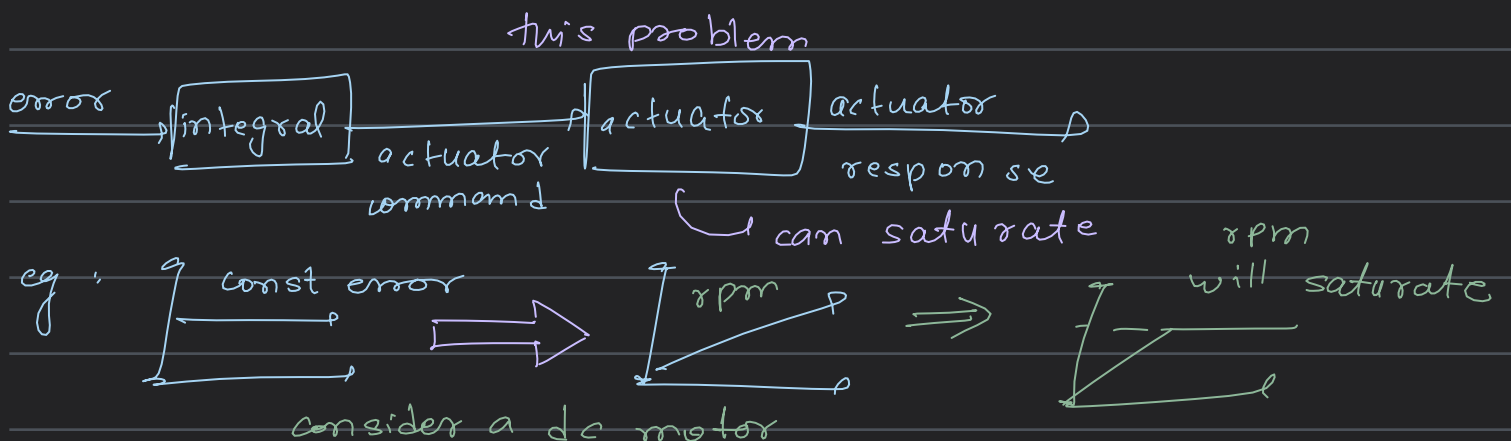
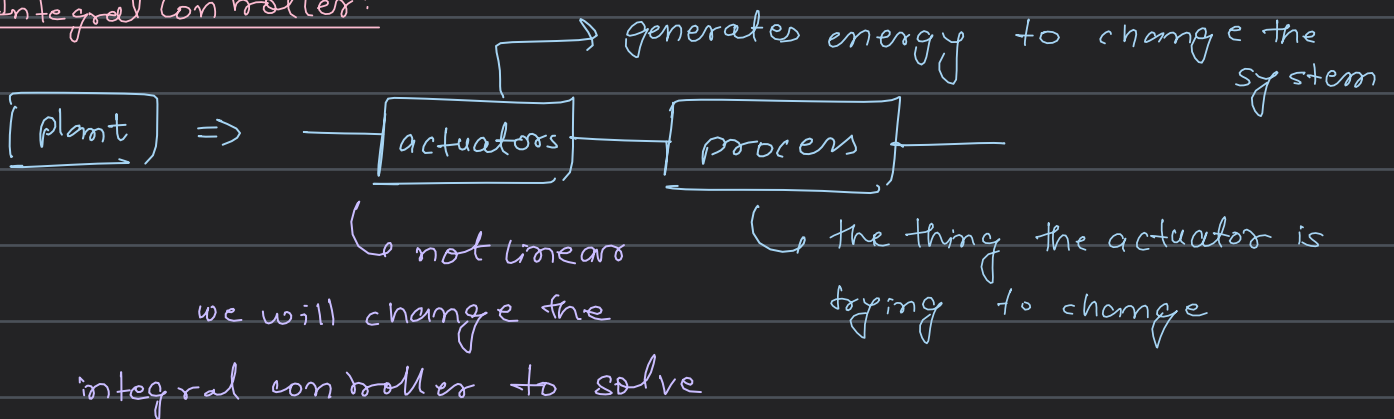
extra speed
this leads to oscillation around 50m

Proportional-Integral-Derivative controller

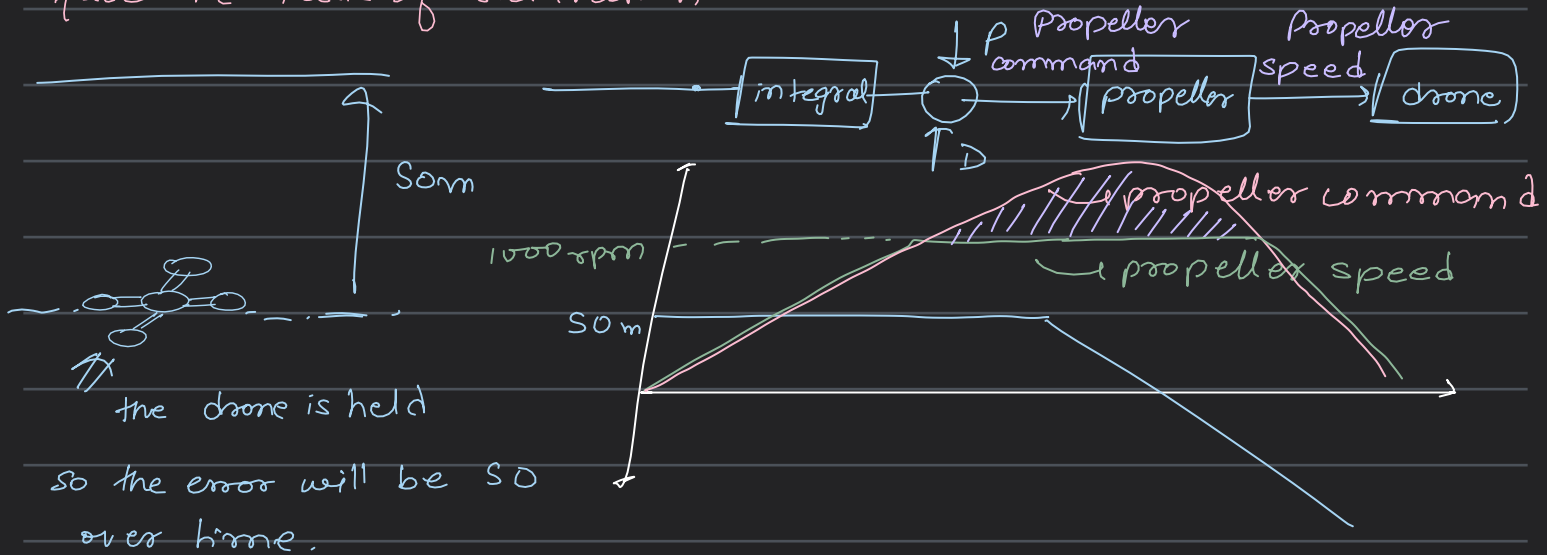


$$u(t) = K_p e(t) + K_I \int_0^t e(t) dt + K_d \frac{de(t)}{dt}$$

Integral Controller:



Only if our actuator system is linear then only we won't face the issue of saturation



As the error dec - propeller command dec as

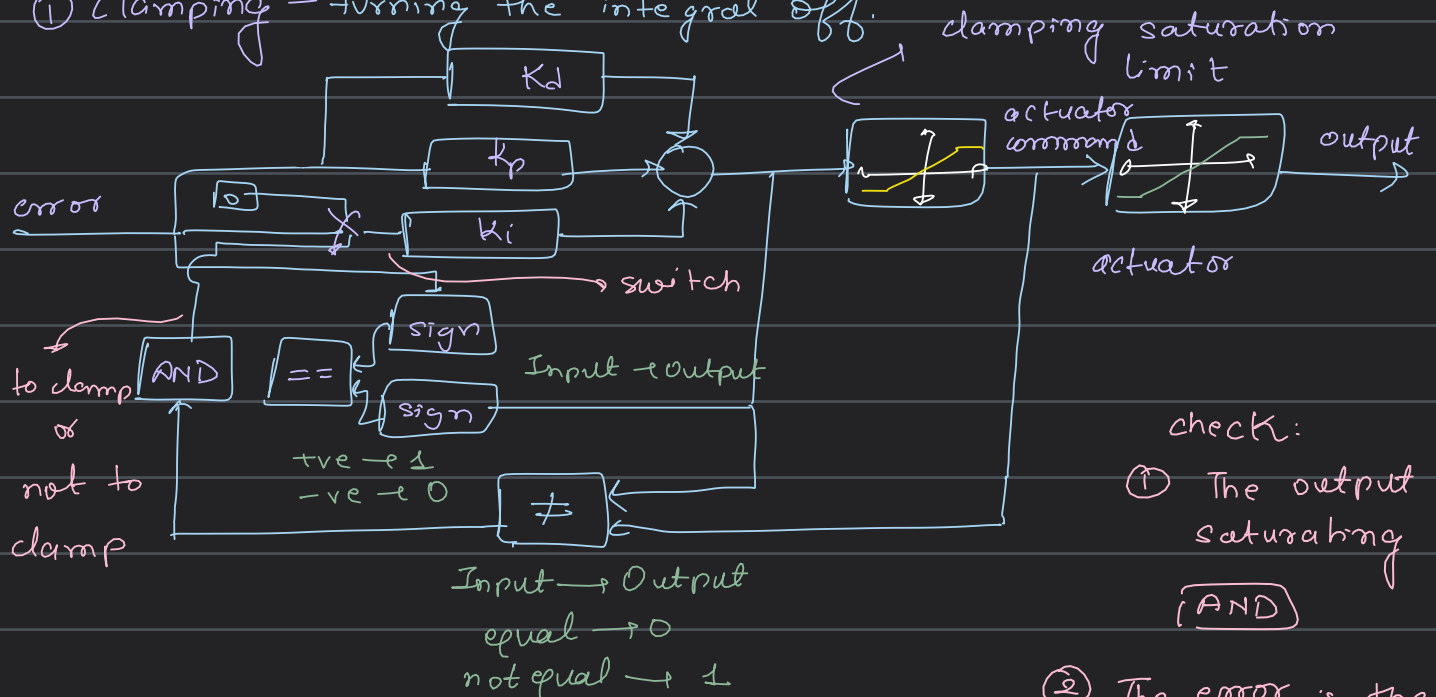
we let go the drone

well untill the propeller command goes from 2000 rpm to 1000 rpm there won't be any actual reduction in propeller speed so the drone will skyrocket even with dec errors

integral windup

Integral antiwindup - keep the integrated value from inc past some specified limit

① Clamping - turning the integral off.



check:

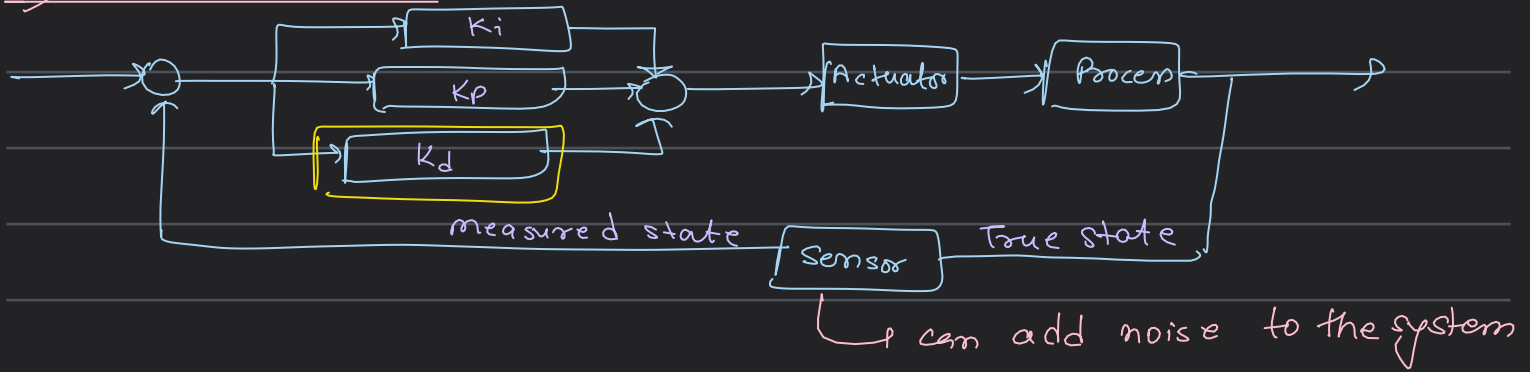
① The output is saturating

(AND)

② The error is the

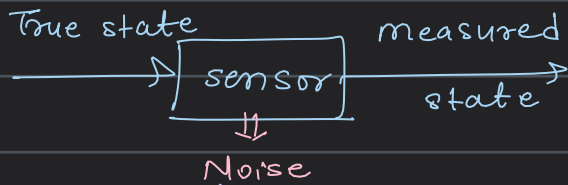
same sign as the controller output

Derivative Controller

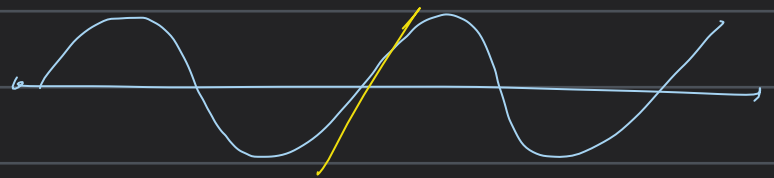


Noise is a random disturbance on a signal (high freq)

low freq noise

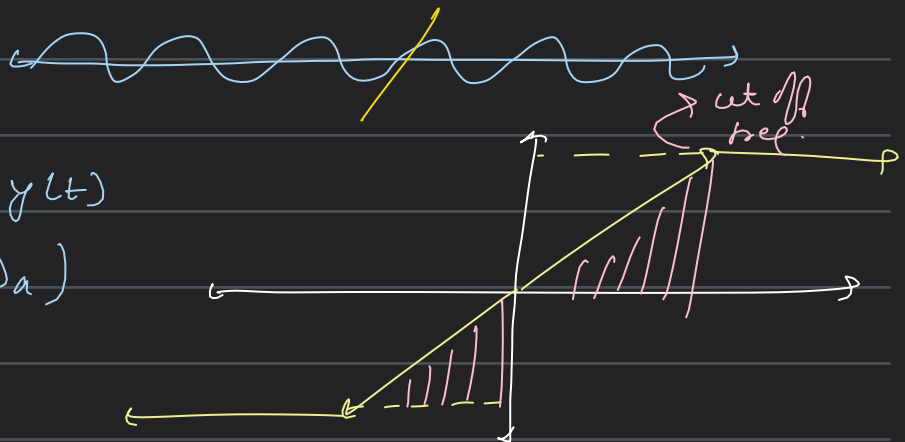


equal I
at diff v
↓
white noise



high freq noise

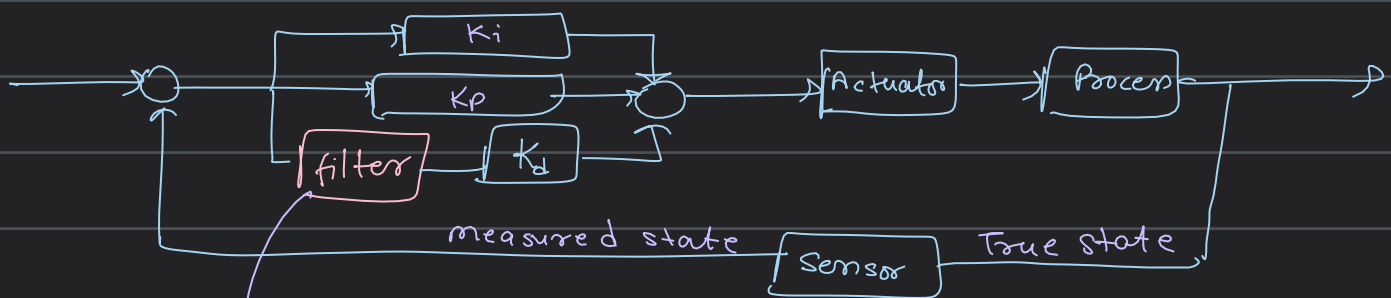
with same amp
high freq noise
will have higher
derivative



$$\text{noise} = A \sin(\omega_a t + \phi_a) = y(t)$$

$$\frac{\partial y(t)}{\partial t} = A \omega_a \cos(\omega_a t + \phi_a)$$

⊗ we will add a filter that will block freq above a certain point from entering the derivative controller



remove the noise beyond a certain freq

so that it will not affect the derivative controller much

Noise - low amp

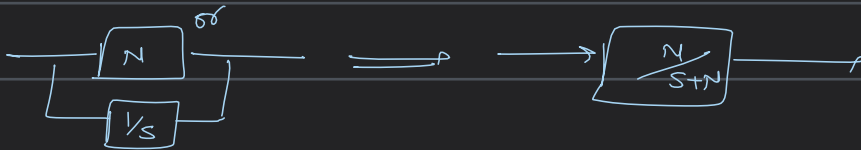
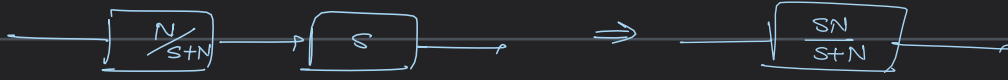
Signal - high amp & low amp
+ low freq (high freq)

→ remove this with the help of a low pass filter

for Laplace domain

low pass filter with cut off at N rad/s

$$\frac{N}{s+N} \rightarrow \frac{1}{s/N + 1} = \frac{1}{z+1}$$



PID Tuning

well behaved system $\rightarrow$ $\frac{N}{s}$

- non-linear
- open loop unstable

→ advanced methods

- huge delay
- non-minimum phase

yes

you have

you have a

physical hardware

model

design with a physical system

model based tuning

P - inc P reduces rise time

I - inc I improves stability & dec overshoot

D - inc D improves stability & dec overshoot

tweak gains manually

sum input sep

derive model from first principle

writing eqⁿ in terms of derivatives

system identification

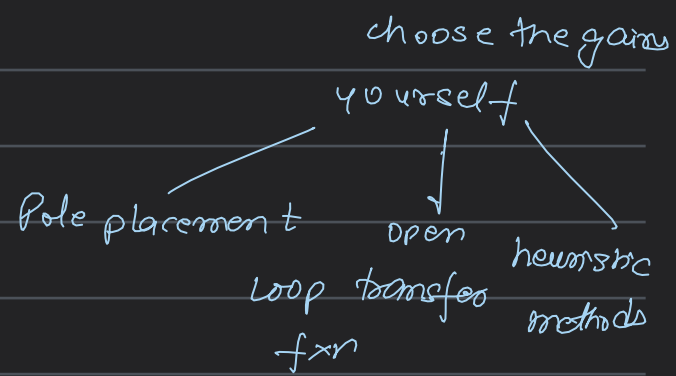
(step / measured response)

(eg: transfer fn)

tuning method

automatic tuning

manual tuning



```

class PIDController:
    def __init__(self, Kp, Ki, Kd):
        self.Kp = Kp
        self.Ki = Ki
        self.Kd = Kd
        self.integral_error = 0
        self.previous_error = 0
    def update(self, error, dt):
        P_term = self.Kp * error

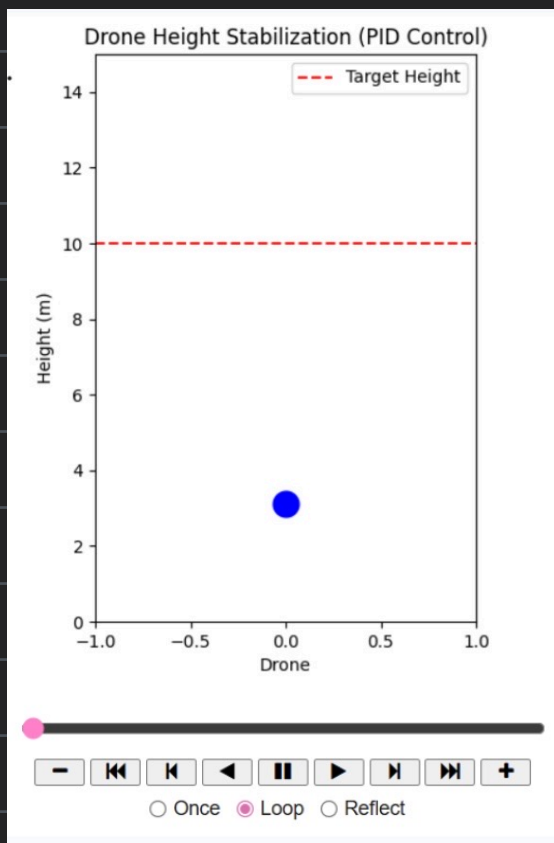
        self.integral_error += error * dt
        I_term = self.Ki * self.integral_error

        derivative_error = (error - self.previous_error) / dt
        D_term = self.Kd * derivative_error

        self.previous_error = error

        thrust = P_term + I_term + D_term
        return thrust

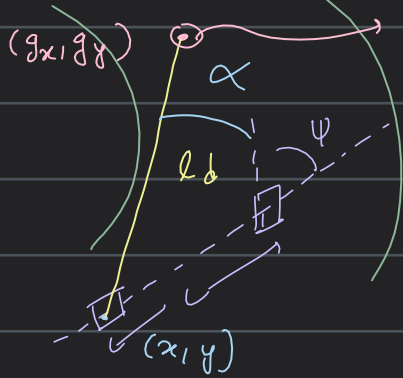
Kp = 5.0
Ki = 10.0
Kd = 6.0
  
```



Pure Pursuit Steering Controller:

to compute the steering angle that moves the rear axle of the vehicle toward a goal point on the path that is a fixed look-ahead distance ahead of the vehicle.

Deriving the steering Angle (δ):



target point

$$\alpha = \tan^{-1} \left(\frac{g_y - y}{g_x - x} \right) - \psi$$

from the bicycle model:

$$\delta = \tan^{-1} \left(\frac{2L \sin \alpha}{l_d} \right)$$

Key Tuning parameter - Lookahead Distance (l_d)

small: $\delta \rightarrow$ large
(tight track)

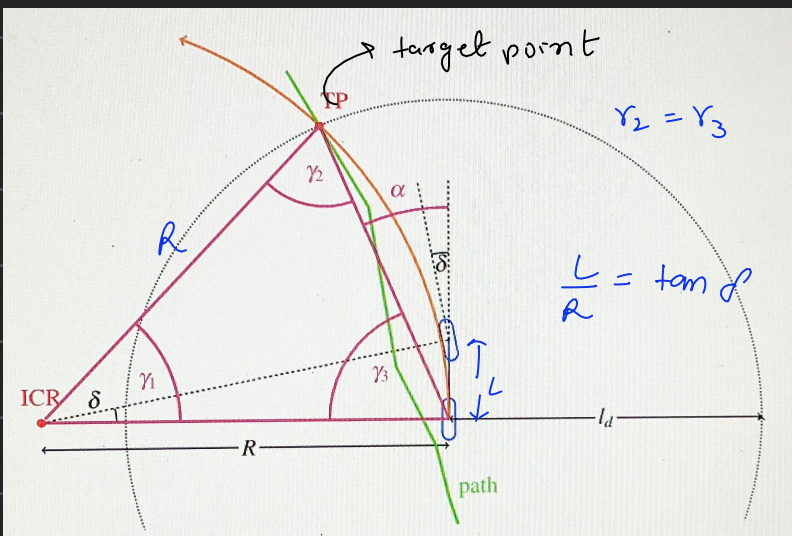
large: $\delta \rightarrow$ small
(smooth steering)

$$l_d = K_{dd} \cdot v + l_{d, \min}$$

gain

minimum lookahead to prevent instability at low speeds

Derivation:



R : instantaneous centre of rotation

$$\gamma_3 + \alpha = 90^\circ \Rightarrow \gamma_2 = \gamma_3 = 90^\circ - \alpha$$

$$\gamma_1 = 2\alpha$$

from sine law,

$$\frac{l_d}{\sin 2\alpha} = \frac{R}{\sin 90^\circ - \alpha}$$

$$\Rightarrow l_d \cos \alpha = 2R \sin \alpha \cos \alpha$$

$$R = l_d / 2 \sin \alpha \quad (1)$$

$$L/R = \tan \delta \quad (2)$$

from (1) & (2)

$$\delta = \tan^{-1} \left(\frac{2L \sin \alpha}{l_d} \right)$$

